

ADVANCE PROGRAMMING TUTORIAL

I/O STREAMS

There are 2 types of I/O streams in Java:

- 1) ByteStreams
- 2) Character Streams

The general rule of thumb to identify which classes belong to which type of Stream is to check the last word of the class name. ByteStream classes typically end with words Stream (eg: BufferedInputStream, DataInputStream, etc.). Character Stream classes tend to end with Reader/Writer (eg: FileReader, FileWriter, etc.)

Let us see an example of ByteStreams:

Q1) In this question we would like to read and write to files byte by byte. For this we will make use of predefined classes FileInputStream and FileOutputStream. In your main() method, initialize one object of each of these classes to null. We will copy the contents of a file named sourcefile.txt to targetfile.txt. To do this, you first need to create sourcefile.txt in the same directory as your Java program with some dummy text. Now coming back to the Java program pass "sourcefile.txt" as argument to the constructor of FileInputStream, and "targetfile.txt" to constructor of FileOutputStream. Note that targetfile.txt does NOT have to be pre-created like sourcefile.txt. If the file is not found, then it will be created for you. Now use a while loop to read sourcefile.txt byte by byte. Store this byte in a temp variable, and write it to targetfile.txt in the body of the loop. When the while loop is excited, close the 2 streams, and run the program. You will NOT see any output on the console, but the targetfile.txt would be created, which would have the same contents that you would have written in sourcefile.txt.

[Note to TAs: Please ask students to create sourcefile.txt before moving onto the code.]

Solution:

```
// Java Program illustrating the
// Byte Stream to copy
// contents of one file to another file.
import java.io.*;
public class Classroom2 {
    public static void main(String[] args) throws IOException{

        FileInputStream sourceStream = null;
        FileOutputStream targetStream = null;
```

```

    try {
        sourceStream = new FileInputStream("sourcefile.txt");
        targetStream = new FileOutputStream("targetfile.txt");

        // Reading source file and writing
        // content to target file byte by byte
        int temp;
        while ((temp = sourceStream.read()) != -1) {
            targetStream.write((byte) temp);
        }
    }
    finally {
        if (sourceStream != null)
            sourceStream.close();
        if (targetStream != null)
            targetStream.close();
    }
}
}

```

Now we will consider the use of Character Streams.

Q2) The work here is similar to Q1. We will read from a source file test.txt, character by character, and print each character on the console. Like last time, first create test.txt in the same directory as your Java program, with some dummy text. Then instantiate an object of the FileReader class, passing test.txt as the argument. Now, again, use a while loop to read the contents till the end of the file, but instead of writing to another file, this time just print the temp character on the console. Do not forget to close the FileReader stream object once the reading operation is completed.

Solution:

```

import java.io.*;

public class Classroom2 {
    public static void main(String[] args) throws IOException {
        FileReader sourceStream = null;
        try {
            sourceStream = new FileReader("test.txt");

```

```

        // Reading sourcefile and
        // writing content to target file
        // character by character.
        int temp;
        while((temp = sourceStream.read()) != -1){
            System.out.print((char)temp);
        }
    }
    finally {
        // Closing stream as no longer in use
        if (sourceStream != null)
            sourceStream.close();
    }
}
}

```

Some points to note:

- 1) Do throw the IO Exception when using I/O streams, and close the streams that you create for these operations.
- 2) The ByteStream reads 8 bit bytes from binary format. Character streams read 1 character, which are represented in Java using the 16 bit Unicode format.

Here are some references for more information:

- 1) [Java IO : Input-output in Java with Examples - GeeksforGeeks](#)
- 2) [Java IO - javatpoint](#)